

# Cloudbeds Senior Backend

## Technical Interview Prep - Consolidated

4-day plan • 60 minutes/day • Integrations team • 60-minute technical interview • your real IRIS/WHOIS project stories

### Bottom line

Your pasted notes are directionally good, but a few claims are too specific. The exact role clearly emphasizes partner APIs, distributed/event-driven workflows, idempotency, retries, dead-letter handling, reconciliation, PostgreSQL, AWS/Kubernetes, observability and production incident response. GraphQL and real-time webhooks are confirmed Cloudbeds API surfaces. Kafka/on-call exist in other current Cloudbeds roles, but are not explicitly confirmed for this exact Integrations posting.

## Reality check: what is actually confirmed?

| CONFIRMED - exact role                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | LIKELY / company-level                                                                                                                                                                                                                                                                                                                                                 | DO NOT ASSUME                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Partner APIs, marketplace and connectors to accounting, POS, revenue management, door locks, channels and compliance systems.</li><li>• Python / Java and React in the codebase.</li><li>• PostgreSQL, caching, API design, distributed systems and async/event-driven workflows.</li><li>• Idempotency, retries, dead-letter handling and reconciliation.</li><li>• Terraform, Docker, Kubernetes, Helm, Argo CD, AWS, CI/CD, observability and production incident response.</li></ul> | <ul style="list-style-type: none"><li>• Public API offers REST-first services, GraphQL support and real-time webhooks.</li><li>• Cloudbeds uses messaging/streaming technology elsewhere; another current backend role names Kafka and gRPC.</li><li>• Cloudbeds engineering uses on-call rotations in some teams (current backend / SRE / DevOps postings).</li></ul> | <ul style="list-style-type: none"><li>• "Hundreds of thousands of concurrent API requests" - no public source confirms that number for this team.</li><li>• Kafka or RabbitMQ specifically for Integrations - exact broker is not in your posting.</li><li>• An on-call rotation for this exact team - incident response is confirmed, rotation is not.</li><li>• JWT as Cloudbeds auth - official docs emphasize API keys and OAuth 2.0.</li></ul> |

### Is on-call realistic?

**Yes, at Cloudbeds generally.** Current Cloudbeds engineering postings explicitly mention on-call rotations, and your exact role requires production incident response. But the exact Integrations posting does not say "on-call rotation", so do not assume the schedule.

*Ask: "Is the Integrations team part of an on-call rotation? How often, what typically pages the team, and how is coverage handled across time zones?"*

### Important correction on "huge API traffic"

Cloudbeds is a large global platform, but its public partner API is rate-limited. Their docs currently describe low per-property/partner request limits and 429 handling. The interview lesson is not "memorize massive QPS"; it is "design integrations that respect rate limits, use webhooks/events, batch where possible, retry safely, and reconcile."

# Day 1 - API design + integration boundaries

Goal: design a clean partner integration and explain every trade-off without rambling.

## 20 min - Learn / refresh

- REST resources, verbs and status codes: 200/201/202/204, 400/401/403/404/409/422, 429, 502/503.
- Pagination, versioning, backwards compatibility, timeouts, rate limits and request IDs.
- Authentication: API keys vs OAuth 2.0. Know JWT conceptually, but do not call it Cloudbeds-specific.
- REST vs GraphQL: resource-oriented simplicity vs flexible client-selected queries.

## 25 min - Practice one design

- Design: "Cloudbeds must sync reservations to a third-party accounting / POS system."
- Clarify source of truth, direction of sync, latency requirement and failure semantics.
- Decide sync API vs event/webhook + async worker.
- Add rate limiting, idempotency, retries, DLQ and reconciliation.
- Describe metrics, logs, tracing and operational ownership.

## 15 min - Say this out loud

1. First I would clarify the data contract and which system is authoritative.
2. For user-facing reads/writes I would keep synchronous calls narrow; for propagation to partners I would prefer asynchronous processing where eventual consistency is acceptable.
3. Every externally visible side effect needs an idempotency strategy because retries and duplicate delivery are normal.
4. I would cap retries with exponential backoff + jitter, honor 429/Retry-After, and route permanent failures to a DLQ or failure state.
5. I would add reconciliation so missed or out-of-order events do not become permanent data drift.

## Cloudbeds-specific fact worth knowing

Their public API supports REST-first services, GraphQL support, real-time webhooks, OAuth 2.0 and API-key based access. Webhooks are important because the public API is rate-limited; event-driven updates avoid wasteful polling.

## REST vs GraphQL - interview answer

- REST: predictable resource model, simple HTTP semantics/caching, good for stable integration contracts.
- GraphQL: useful when clients need different shapes or want to avoid multiple round trips; requires complexity controls, authorization at field/resolver level and query-cost protection.
- Do not answer "GraphQL is better." Answer based on consumers, contract stability, caching, observability and operational complexity.

## Authentication - what to say

- API key: simple server-to-server credential; rotate/store securely; scope it; never expose it client-side.
- OAuth 2.0: delegated authorization; useful when a property/user authorizes a partner app; handle scopes, token lifecycle and revocation.
- JWT: token format, not an authorization protocol by itself. Know signature/expiry/audience/issuer basics, but do not claim Cloudbeds uses JWT for this API unless they tell you.

## Day 2 - Distributed systems + resiliency

Highest-value day. The exact role explicitly names idempotency, retries, dead-letter handling and reconciliation.

### 20 min - Core concepts

- At-least-once delivery -> duplicates are expected.
- Idempotency keys / event IDs + durable deduplication.
- Ordering: do not assume events arrive in order.
- Exponential backoff + jitter; retry only transient failures.
- Dead-letter queues / failure stores for poison or exhausted messages.
- Reconciliation jobs to detect and repair drift.
- Circuit breaker to fail fast when a dependency is unhealthy.
- Transactional outbox to bridge DB commit and event publication.

### 25 min - Scenario

- "A partner API is down for 3 hours while reservation events continue arriving."
- Persist events durably; do not hold request threads open.
- Process asynchronously with bounded concurrency.
- Back off retries; stop hammering the partner; respect rate limits.
- Keep operations idempotent so replay is safe.
- Observe queue depth/oldest-message age/error rate.
- After recovery, drain gradually and run reconciliation.

### 15 min - Strong 2-minute answer

*"I would treat partner failure as normal, not exceptional. I would accept or persist the event first, then process it asynchronously. Transient failures get bounded retries with exponential backoff and jitter; permanent failures go to a DLQ/failure state. Replays must be idempotent. I would monitor backlog age and failure rate, and after recovery use reconciliation against the authoritative source to catch anything missed or delivered out of order."*

### Concept cheat sheet

| Term                        | What you should be able to explain                                                                                                                                                                                   |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Idempotency</b>          | Same logical request/event can be processed multiple times without producing duplicate side effects. Typical implementation: stable idempotency/event key + unique DB constraint/state + return/ack previous result. |
| <b>Retry</b>                | Use for transient failures (timeouts, 429, some 5xx). Exponential backoff + jitter + max attempts. Do not blindly retry validation/auth errors.                                                                      |
| <b>Circuit breaker</b>      | After repeated dependency failures, open the circuit and fail fast. After a cooldown, allow limited half-open probes. Protects threads/connections and prevents cascading failure.                                   |
| <b>DLQ / failure store</b>  | Keeps messages that cannot be processed after policy is exhausted. Must have alerting, inspection and replay/remediation workflow; it is not a trash bin.                                                            |
| <b>Reconciliation</b>       | Periodic comparison with an authoritative source to detect missing, stale or divergent state and repair it. Essential when events can be lost, duplicated or reordered.                                              |
| <b>Transactional outbox</b> | Write the business change and an outbox record in the same DB transaction; a separate publisher emits it. Avoids "DB committed but message publish failed".                                                          |
| <b>Eventual consistency</b> | Systems may temporarily disagree, but converge. You need observable lag, deterministic conflict rules and reconciliation.                                                                                            |
| <b>Exactly-once</b>         | Be careful with the phrase. End-to-end exactly-once is hard; most practical systems combine at-least-once delivery with idempotent processing.                                                                       |

# Cloudbeds integration behavior worth understanding

These are useful concrete examples from Cloudbeds public developer documentation.

## Webhooks are not “fire once and forget”

Cloudbeds documents webhook retries on failed/non-2xx delivery and recommends returning 2xx quickly, then doing work asynchronously. Their docs also warn that event ordering is not guaranteed; duplicate delivery is possible in timeout/retry situations. This is exactly why idempotency and reconciliation matter.

## Webhook receiver pattern

- Authenticate/validate request as appropriate.
- Record event ID/type/timestamp/property context durably.
- Return 2xx fast - do not wait for slow downstream work.
- Queue async processing.
- Dedupe using stable identifiers / state.
- Handle out-of-order updates using version/timestamp/domain rules where possible.
- Alert on repeated failures; support replay.

## Rate limits / 429

- Design with a client-side rate limiter and bounded concurrency.
- Honor Retry-After when provided.
- Batch requests where the API supports it.
- Prefer webhooks over frequent polling.
- Distinguish 429/503/timeouts (potentially retryable) from 400/401/403/422 (usually fix input/auth first).

## “Ugly integrations” are realistic

- External systems may have incomplete docs, inconsistent fields, odd time zones, legacy auth, slow responses or unstable schemas.
- Use an adapter/anti-corruption layer so partner quirks do not leak into your core domain.
- Normalize external payloads into internal models; validate aggressively at the boundary.
- Capture partner-specific metrics and error taxonomy so support is diagnosable.
- Build replay/reconciliation tools because manual recovery will otherwise become the operational model.

## Do not over-focus on Kafka vs RabbitMQ

For this interview, first explain the delivery semantics and failure behavior you need. Then say the specific broker depends on the platform. Kafka is confirmed in another current Cloudbeds backend role, but your Integrations posting only requires asynchronous/event-driven workflows - not a named broker.

## Likely design follow-up: prevent overbooking / stale availability

1. Identify the authoritative inventory/reservation state and which operations require stronger consistency.
2. Use atomic DB constraints/transactions for local inventory changes where possible.
3. Propagate updates to external channels asynchronously but with low-latency event processing.
4. Make updates idempotent and version-aware so retries/out-of-order messages do not overwrite newer state.
5. Use reconciliation to find drift between Cloudbeds and channels; alert on lag/backlog.
6. Do not claim zero overbooking is guaranteed purely by “real-time sync” - network partitions and partner behavior still exist.

## Day 3 - PostgreSQL + cloud operations + incidents

Goal: show that you can own production, not just write application code.

### 20 min - PostgreSQL / data

- Indexes: what they speed up, write/storage cost, composite index order.
- Unique constraints as a durable idempotency guard.
- Transactions and isolation at a practical level.
- Locking / contention / long transactions.
- N+1 queries, EXPLAIN/slow-query diagnosis.
- Connection pools and exhaustion.
- Caching: TTL, invalidation, stampede and stale reads.

### 20 min - Cloud / operations

- Kubernetes: deployments, pods, readiness/liveness, resource limits, autoscaling, rollback.
- AWS at architecture level: compute, managed DB, queues/events, storage, IAM/networking.
- Terraform / Helm / Argo CD: repeatable infrastructure + declarative deployment / GitOps.
- Observability: metrics + structured logs + traces + correlation/request IDs.
- Know how you would mitigate before root-causing.

### 20 min - Production incident drill

Question: "API p95 latency jumped from 200 ms to 3 s after a release. What do you do?"

1. Confirm scope/impact: endpoint, region, tenants, error rate, saturation.
2. Check dashboards + recent deploy/config/schema/infrastructure changes.
3. Compare latency breakdown: app, DB, cache, external dependency, queue.
4. Look at traces/logs; inspect CPU/memory/thread or connection pool saturation.
5. Mitigate: rollback, disable feature, reduce concurrency, shed load, scale safely.
6. After stabilization: root cause, tests/guardrails, alerting/runbook and follow-up action.

### Senior signal

Do not jump straight to "restart pods". Start with impact and evidence, mitigate safely, then investigate. A senior answer separates restoration of service from root-cause analysis.

### One real incident story to prepare

- Situation: what production behavior was wrong and why it mattered.
- Signal: how you detected it - alert, user report, metric, logs.
- Diagnosis: what hypotheses you tested and what evidence narrowed it down.
- Mitigation: what you changed immediately to reduce impact.
- Permanent fix: code/config/architecture change.
- Prevention: test, metric, alert, runbook, capacity rule or design change.

## Your strongest recent integration story - IRIS + WHOIS

This is your best evidence for the Cloudbeds role because it is not a textbook example: it is a real third-party integration pipeline with unreliable external providers, synchronous + asynchronous paths, caching, Kafka, idempotent delivery, rate limits, schema drift and production failure handling.

### 60-second umbrella version

*"One of my latest areas of work was our third-party integration service, IRIS. Multiple reporting APIs need the same external reputation, passive-DNS and WHOIS data, so instead of each API owning separate clients and parsing logic, they call one internal integration service. IRIS fans out to providers asynchronously, returns whatever fresh data is needed synchronously, and also publishes results to Kafka or RabbitMQ so the delivery databases are updated for future reads. The reporting APIs are DB-first and only call IRIS when data is missing or stale, which protects external providers and keeps latency predictable. My recent WHOIS work had two separate tracks: IP WHOIS and domain WHOIS."*

### High-level architecture you should be able to draw

Reporting API → check delivery DB / fetch timestamp → if fresh, serve stored data.

If missing/stale → POST IRIS / fetch with entity type + requested sources.

IRIS fans out to third-party sources asynchronously and isolates per-source failures.

Fresh result can be returned immediately to the caller.

The same result is published to Kafka/RabbitMQ → delivery writer → DB for future requests.

Result: synchronous freshness for the current request + asynchronous self-healing of the cache/read model.

### Why this architecture exists

Centralizes credentials, provider-specific clients, parsing and publishing instead of duplicating them across every reporting API. Keeps reporting APIs insulated from third-party quirks - the same anti-corruption / adapter-layer idea that matters in Cloudbeds partner integrations.

DB-first behavior limits calls to rate-limited providers and lets reports degrade to stored/stale data when an external source or IRIS is unavailable.

Async fan-out means independent providers can be queried concurrently; one failed source does not have to destroy the whole response.

## Project 1 - IP WHOIS

Treat this as a separate project story from domain WHOIS. Keep the first answer simple, then go deeper only when they ask.

### 90-second answer

*"For IP WHOIS we integrated registry data using the Python ipwhois library. The interesting part was not just calling the provider - it was fitting the result into a reliable distributed pipeline. We parse registry responses into our common objects model, publish them to Kafka, and the delivery service stores them by IP range. The cache/fetch-date model is range-aware, so if an address belongs to a recently fetched range we can serve the stored WHOIS instead of hitting the registry again. On replay, the delivery side merges by range and keeps the newest relevant objects, so old messages do not resurrect stale data."*

### Technical points to remember

IP WHOIS uses ipwhois in Python with configured timeout/retry behavior.

Data is grouped by IP network/range rather than treating every IP as an unrelated key.

Fetch-date indexes are range-based for IPv4/IPv6, reducing unnecessary repeated registry calls.

Delivery merge logic chooses the newest object for a range; replays are safe instead of blindly appending duplicates.

This is a good example of data modeling + idempotency + external rate-limit awareness, not just an API client.

### Likely follow-ups

Why cache by range instead of exact IP? → WHOIS ownership is commonly defined for a network range; one fetch can satisfy many addresses in that range.

What happens on duplicate Kafka delivery? → deterministic keys / merge-by-newest behavior keep replay from creating incorrect duplicate state.

What if the registry is down? → IRIS logs the per-source failure; callers can still serve delivery-DB data where available.

## Project 2 - Domain WHOIS

This is probably your strongest "messy external integration" story because the protocol and data are inconsistent by nature.

### 90-second answer

*"Domain WHOIS was a separate piece of work. We initially evaluated RDAP, but the data/referral coverage was not sufficient for our use case, so the final path used a Go WHOIS library exposed as a shared object and called from Python through ctypes. The Go layer handles the raw network protocol, referrals and timeouts, while Python owns marshalling and parsing. The hard part was normalizing non-standard registry text: different sections, bilingual responses, redacted values and many timestamp formats. We also added parent-domain fallback for subdomains and kept the Kafka/DB path idempotent so the same WHOIS history can be replayed safely."*

## Technical points to remember

Go shared library (libwhois.so) is called from Python via ctypes; binary is delivered through Artifactory/RPM and deployed with the service.

WHOIS is raw TCP/43 text with registry-specific quirks; parser handles comments, sections, bilingual output, redaction and many timestamp formats.

Parent-domain fallback walks toward the registered domain when a subdomain has no direct record.

Records without a parseable timestamp are rejected because timestamp ordering is part of the history/storage contract.

Domain history is bounded (up to 10 records) and deduplicated by key/timestamp; re-delivery writes the same logical record rather than creating uncontrolled duplicates.

The design chose richer WHOIS data over RDAP simplicity - a concrete trade-off, not "newer protocol is always better."

## Likely follow-ups

Why Go + ctypes instead of all Python? → reuse a stronger protocol/referral implementation while keeping the service integration in Python; trade-off is binary lifecycle/deployment complexity.

How do you trust malformed external data? → tolerant parsing at the boundary, explicit redaction handling, timestamp validation, and reject/alert instead of guessing.

What if response format changes? → parser is deliberately tolerant; unknown mappings/invalid shapes should surface via metrics/Sentry rather than silently corrupting data.

## IRIS failure modes - perfect interview material

Third-party source timeout/down: isolate per-source exception, return partial results, bound latency with per-source timeouts.

IRIS down: callers degrade to delivery-DB data; stale data is better than a total report failure.

Delivery consumer down: Kafka retains backlog; consumer replays later; idempotent writes make recovery safe.

Kafka publish failure inside IRIS: current design can fail the fetch/publish path and lose that freshly fetched write-path result - a real gap you can openly identify.

Rate-limited provider: fetch timestamps/TTL reduce call volume; one known improvement is better backoff for rate-limited attempts rather than recording only successful fetches.

Schema/category drift: fail visibly, alert, and update mappings instead of silently storing a wrong classification.

## A senior-level way to discuss gaps

*"I try not to present existing architecture as perfect. For example, one thing I would improve is making the publish path more durable - an outbox or persisted retry state would avoid losing freshly fetched data if Kafka publishing fails. I would also centralize some duplicated cache/backoff policy so every caller does not have to reimplement it."*

## How your IRIS work maps directly to Cloudbeds

Cloudbeds problem: partner APIs you do not control. Your experience: many external providers with different protocols, timeouts, limits and response quirks.

Cloudbeds problem: idempotency / replay. Your experience: deterministic Kafka keys, timestamp-based dedupe/merge and replay-safe delivery.

Cloudbeds problem: retries / partner outages. Your experience: per-source failure isolation, timeouts, cached fallback and backlog replay.

Cloudbeds problem: reconciliation / eventual consistency. Your experience: DB-first read model + async write path; future reads converge on stored results.

Cloudbeds problem: rate limits. Your experience: 24h fetch TTLs, range-aware IP caching and explicit provider backoff behavior.

Cloudbeds problem: ugly legacy integrations. Your experience: raw WHOIS protocol, Go/Python boundary, inconsistent registry text and tolerant adapter/parsing logic.

## Use this when they ask "Why are you a good fit for Integrations?"

*"The reason this role feels close to my recent work is that I already spend a lot of time at boundaries between systems we do not control. IRIS has to deal with unreliable providers, rate limits, inconsistent contracts, partial failures, async delivery and replay. The WHOIS work in particular forced us to make messy external data behave predictably inside our own system. That is the same class of engineering problem I understand Cloudbeds Integrations deals with."*

## Do not over-explain the project

Start with the 60-90 second architecture story. Do not open with protobuf keys, ctypes lifecycle or parser edge cases.

If they ask "what did YOU do?", switch immediately from "we" to your exact decisions/implementation areas.

Separate IP WHOIS and domain WHOIS. They share a pipeline, but the technical problems are different.

Do not call the system exactly-once. Say replay-safe / idempotent and explain the concrete key/merge behavior.

Mention known gaps when relevant. Recognizing failure modes is stronger than pretending the system is flawless.

## Day 4 - Mock technical interview + your stories

Do this on Monday. Speak answers out loud. Aim for 2-3 minutes each unless they pull you deeper.

## 60-second technical intro

*"I have almost seven years in backend engineering. I started in a large Java/Spring legacy environment and gradually moved into microservices, cloud systems and distributed/event-driven backend work. More recently I have been working primarily in Python, and I am comfortable owning work from technical design through implementation, deployment, observability and production support. The part of this role that fits me especially well is building reliable integrations and distributed workflows where external systems can fail or behave unpredictably."*

### Prepare 3 project stories

- Legacy / monolith: a difficult change, constraint or migration and how you reduced risk.
- Cloud/microservices: AWS + containers/Kubernetes, service boundaries, deployment and observability.
- Distributed/event-driven: delivery semantics, duplicates, failures, recovery and a concrete trade-off.

### For every story, know

- What was the actual problem?
- What did YOU own / decide?
- What alternatives did you consider?
- What failed or was risky?
- How did you test / observe it?
- What measurable or operational result changed?

## 35 min - Mock questions

1. Design a reliable reservation sync with an external partner that sometimes times out.
2. How would you make a consumer idempotent?
3. What do you do when a partner returns 429 or 503?
4. Events arrive out of order. How do you prevent an old update from overwriting a new one?
5. What happens if your DB transaction commits but publishing the event fails?
6. When would you choose async processing instead of a synchronous API call?
7. How do you evolve an API without breaking external partners?
8. How do you investigate a slow API?
9. When is caching dangerous?
10. Tell us about a production incident you owned.
11. Tell us about a technical disagreement and how the decision was made.
12. How would you decide between a queue and a stream/log-style platform?

# Fast answer patterns

Use these as structures, not scripts. Start simple, then let them ask deeper questions.

| Question/theme                 | Answer skeleton                                                                                                                                                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| External API is failing        | Timeouts + bounded concurrency -> classify error -> retry transient failures with backoff/jitter -> circuit breaker if sustained -> DLQ/failure state after policy -> metrics/alerts -> reconciliation.    |
| Idempotent write               | Client/event provides stable key -> store key/result atomically (often unique constraint) -> duplicate returns existing result / no duplicate side effect -> define retention and scope.                   |
| Out-of-order event             | Carry version/sequence/timestamp where trustworthy -> compare against current state -> ignore stale update or apply deterministic merge rule -> reconcile when order cannot be trusted.                    |
| API versioning                 | Prefer additive backward-compatible changes -> tolerate unknown fields -> deprecate with telemetry and clear migration window -> version only when breaking semantics truly require it.                    |
| Cache                          | Cache only when staleness is acceptable -> define invalidation/TTL -> protect against stampede -> observe hit rate and stale-data impact -> source of truth remains elsewhere.                             |
| Queue vs stream                | Queue: work distribution/competing consumers. Stream/log: retained ordered history + multiple independent consumers/replay. Real systems overlap; choose based on semantics, retention and consumer model. |
| Strong vs eventual consistency | Use strong consistency for invariants that cannot temporarily diverge; use eventual consistency for cross-system propagation where availability/decoupling matters, with visible lag and reconciliation.   |
| Circuit breaker                | Failure threshold -> open/fail fast -> cooldown -> half-open probes -> close on recovery. Pair with timeouts and retry budgets; breaker alone does not recover data.                                       |

## Optional coding side-track - 20 to 30 min/day

- Python: dict/set, parsing, queues, simple BFS, clean functions and edge cases.
- Write a tiny idempotent event processor: event\_id -> process once -> retry transient errors -> failure store.
- SQL: joins, GROUP BY, indexes, uniqueness and a query-plan discussion.
- Do not spend your limited prep time on obscure dynamic programming unless they explicitly told you the interview is LeetCode-heavy.

### If you only remember five things

1) idempotency, 2) retries/backoff/jitter, 3) partner/API failure handling, 4) one strong distributed-system design story, 5) one strong production-incident story.

## Questions to ask them + final checklist

Pick 3-4. These are useful to you and also reveal how mature the system actually is.

1. What failure modes cause the most pain in the Integrations platform today?
2. How are partner outages handled today - retry queues, DLQs, reconciliation, manual tooling?
3. What messaging/event technology does this team actually use, and what delivery guarantees do you design around?
4. How much of the work is new integrations versus improving the platform underneath them?
5. Is the Integrations team part of an on-call rotation? How often, what typically pages the team, and how is coverage handled across time zones?
6. What does production ownership look like after a senior engineer ships a change?
7. What would you expect me to have improved or owned after the first six months?

### Final 10-minute checklist before the interview

- ☐ 60-second technical intro is natural, not memorized word-for-word.
- ☐ IRIS umbrella story + IP WHOIS + domain WHOIS are ready; each has problem, your decision, trade-off and result.
- ☐ Can explain idempotency using your real Kafka key / merge examples without casually claiming “exactly once.”
- ☐ Can explain 429/503 retry behavior and when NOT to retry.
- ☐ Can design partner outage recovery with backlog + replay + reconciliation.
- ☐ Can explain one Postgres performance issue and one production incident.
- ☐ Know the exact role stack at a high level; do not pretend Kafka/on-call is confirmed for this team.
- ☐ Have 3 questions ready for Dimitrios and Leonardo.

### Sources / grounding

- Exact Cloudbeds Senior Software Engineer - Integrations posting: [open source](#)
- Cloudbeds API overview - REST, GraphQL, webhooks, OAuth / API surface: [open source](#)
- Cloudbeds Developer Hub - Webhooks: [open source](#)
- Cloudbeds Developer Hub - API FAQ / rate limits: [open source](#)
- Cloudbeds Developer Hub - Authentication: [open source](#)
- Current Cloudbeds Workflow backend role - Kafka/gRPC/on-call example: [open source](#)

#### Last reminder

The interviewers are Staff/Senior engineers. They are more likely to care about your reasoning, failure modes, trade-offs and real production experience than whether you remember every framework command. Explain the simple design first; add complexity only when the requirement demands it.

- Your IRIS/WHOIS project notes: IRIS\_INTEGRATION.md (internal project summary supplied for interview prep).